Social Media Analytics and User Activity System

A MongoDB Database Project Report

Name	Harsh Kumar
Branch	CSE
Section	5-C
Roll No.	202401100200149
Course	[Course name]
Submitted to	[Teacher name]
Date	27 October 2026

Abstract

This project builds the database layer of a small social media platform using MongoDB, a document-oriented NoSQL database. The system stores users, posts, comments, likes, follow relationships and user activity logs. It applies document data modeling (embedding and referencing), enforces data quality through JSON Schema validation and unique indexes, and produces analytics such as trending hashtags, top users and daily engagement using the MongoDB aggregation framework. The project is implemented as a set of repeatable mongosh scripts so that the whole database can be rebuilt and demonstrated from a clean state. Performance optimization, transactions and replication are identified as planned extensions.

1. Introduction

Social media platforms generate large amounts of varied, fast-changing data: profiles, posts with different attributes, comments, likes and activity events. Relational tables with fixed columns can handle this, but they often need many joins and rigid schemas.

MongoDB stores data as flexible JSON-like documents (internally BSON), which allows a post, its hashtags and its metadata to live in one document, and which scales naturally as the data grows.

This project uses MongoDB to model such a platform and to analyze its data. The focus is on the database: how the data is designed, how its quality is protected, and how insights are extracted from it. No user interface is built.

2. Objectives

1. Design a MongoDB schema for a social media application, choosing between embedding and referencing for each relationship.
2. Enforce data integrity using JSON Schema validation and unique indexes.
3. Query related data that is spread across collections.
4. Build analytics using the aggregation framework.
5. Provide scripts that rebuild and demonstrate the whole system from scratch.
6. Identify further extensions: performance optimization, transactions and replication.

3. Scope

Implemented in this submission:

- Data model with six collections
- Seed data and repeatable setup scripts
- Schema validation on users and posts
- Unique and supporting indexes
- Cross-collection queries
- Thirteen aggregation pipelines

Planned as future work: index performance testing with explain(), multi-document transactions, and a replica set with failover.

4. Tools and Technologies

Tool	Purpose
MongoDB Community Server	The database server, running locally as a Windows service
mongosh	Command-line shell used to run all scripts
MongoDB Compass	Graphical tool used to inspect collections and documents
Windows PowerShell	Used to run the scripts

The database is named `social_app` and is reached at `mongodb://localhost:27017`. The project runs entirely on a local machine and needs no hosting.

5. Database Design

5.1 Design approach

In MongoDB, documents are designed around the queries an application makes, not around normalized tables. The main application needs were identified first:

#	Question the application asks	Frequency
1	Show a post with its author, like count and comment count	Very high
2	Show the comments of a post, newest first	High
3	Show a user profile with follower and following counts	High
4	Has user X already liked post Y?	High
5	Who follows user X, and whom does X follow?	Medium
6	Analytics: trending hashtags, top users, daily engagement	Periodic
7	Record every user action	Very high write rate

5.2 Collections

Collection	Purpose
`users`	User profiles with followersCount and followingCount counters
`posts`	Posts with author, text, embedded hashtags, likesCount and commentsCount
`comments`	One document per comment, referencing its post
`likes`	One document per like (user and post), a junction collection
`follows`	One document per follow (follower and followee), a junction collection
`activity_logs`	Records of user actions such as login and view_post

Structure of the main documents:

```
users:         { _id, username, city, interests: [...], isVerified, joinedAt,
                 followersCount, followingCount }
posts:         { _id, authorId, authorUsername, text, hashtags: [...],
isPublic,
                 likesCount, commentsCount, createdAt }
comments:      { _id, postId, userId, username, text, likesCount, createdAt }
likes:         { _id, userId, postId, createdAt }
follows:       { _id, followerId, followeeId, createdAt }
activity_logs: { _id, userId, type, postId (optional), timestamp, meta }
```

5.3 Embedding versus referencing

Embedding places related data inside one document. It is fast to read because one query returns everything, and single-document writes are atomic. **Referencing** stores related data in a separate collection and links it using the `_id` of the other document. MongoDB has no automatic foreign keys, so a reference is simply a copy of the other document's `_id` stored in a field, for example the `postId` field of a comment.

The design decisions were:

1. **Comments, likes and follows are separate collections.** Their number is unbounded. A viral post could have hundreds of thousands of comments, and embedding them would approach MongoDB's 16 MB document size limit and make every new comment rewrite a very large document. Separate collections can also be queried independently, for example all comments written by one user.
2. **Hashtags and interests are embedded arrays.** They are small, bounded and always read together with their parent document.
3. **Extended reference.** A post stores `authorUsername` as well as `authorId`, so a post can be displayed without an additional lookup. Only fields that rarely change should be copied this way.
4. **Computed counters.** `likesCount`, `commentsCount`, `followersCount` and `followingCount` are stored so that reads are instant instead of counting documents every time. This is denormalization: the counter and the underlying collection must be kept in sync. Transactions, planned as future work, are the proper tool for guaranteeing that.
5. **Junction collections** for many-to-many relationships. A user can like many posts and a post can be liked by many users, so each like is its own document.
6. **Dates are stored as real Date values**, never as strings, so that date-based analytics work correctly.

6. Implementation

The project is delivered as five scripts, run in a fixed order.

File	Role
`seed.js`	Drops existing collections, inserts sample data and creates indexes
`validation.js`	Applies JSON Schema validators to `users` and `posts`
`validation_tests.js`	Proves the validation rules work by sending invalid and valid documents
`queries.js`	Cross-collection queries on the referenced data
`aggregations.js`	Thirteen analytics pipelines

Run order:

```
mongosh "mongodb://localhost:27017" C:\mongo-project\seed.js
mongosh "mongodb://localhost:27017" C:\mongo-project\validation.js
mongosh "mongodb://localhost:27017" C:\mongo-project\validation_tests.js
mongosh "mongodb://localhost:27017" C:\mongo-project\queries.js
mongosh "mongodb://localhost:27017" C:\mongo-project\aggregations.js
```

Because `seed.js` drops every collection and dropping a collection also removes its validator, `validation.js` must be run again after every re-seed.

6.1 Seed data

The seed script uses small helper functions (`addPost`, `follow`, `like`, `comment`). Each helper inserts the relationship document and then updates the matching counter with `$inc`, which is atomic for a single document. The dataset contains 6 users, 7 posts (one private), 6 comments, 15 likes, 10 follows and 3 activity log entries.

6.2 Indexes and constraints

Index	Type	Purpose
`likes (postId, userId)`	Unique compound	A user cannot like the same post twice
`follows (followerId, followeeId)`	Unique compound	A user cannot follow the same person twice
`comments (postId, createdAt desc)`	Compound	Fast retrieval of a post's comments, newest first

Trying to insert a duplicate like or follow fails with error code 11000 (duplicate key), so the rule is enforced by the database itself and does not depend on application code.

6.3 Schema validation

MongoDB collections accept documents of any shape by default, so a typo such as `follwers` would be stored silently. To prevent this, `$jsonSchema` validators were added with `validationLevel: 'strict'` and `validationAction: 'error'`.

Collection	Rules
`users`	Required: username (string, at least 3 characters), joinedAt (date), followersCount and followingCount (numbers, minimum 0). isVerified must be boolean and interests must be an array of strings.
`posts`	Required: authorId (objectId), text (string), hashtags (array of strings), likesCount (number, minimum 0), createdAt (date).

The validators are applied with the `collMod` command:

```
db.runCommand({
  collMod: 'users',
  validator: { $jsonSchema: { bsonType: 'object',
    required: ['username','joinedAt','followersCount','followingCount'],
    properties: { username: { bsonType: 'string', minLength: 3 }, /* ... */ }
} },
  validationLevel: 'strict',
  validationAction: 'error'
})
```

`validation_tests.js` demonstrates the guardrails. It sends four invalid documents (username too short, date stored as a string, negative follower count, post missing required fields) and one valid document. The script itself contains no checking logic. It only reports what the MongoDB server decides, so every invalid write is rejected by the server with a document validation error (code 121), and the valid write is accepted and then cleaned up.

6.4 Queries on referenced data

Because comments, likes and follows are separate collections, related data is fetched with a second query that uses the stored `_id`. `queries.js` demonstrates:

1. Comments of a post, newest first, using the post's `_id` as `postId`.
2. The followers of a user: find the `follows` documents, collect the follower IDs, then fetch those users with `$in`.
3. Whether a given user has liked a given post.

4. A duplicate like being rejected by the unique index.

7. Aggregation Analytics

The aggregation framework processes documents through a pipeline of stages, each receiving the output of the previous one. The stages used in this project are `$match`, `$group`, `$sort`, `$limit`, `$project`, `$addFields`, `$unwind`, `$lookup`, `$bucket`, `$facet` and `$setWindowFields`, together with the accumulators `$sum`, `$avg`, `$min`, `$max`, `$push` and `$addToSet`. `$match` is placed as early as possible to reduce the data processed by later stages.

#	Analysis	Main stages
1	Trending hashtags	`$match`, `$unwind`, `$group`, `$sort`, `$limit`
2	Top users by likes received	`$group`, `$sort`, `$limit`
3	Daily engagement	`$dateToString`, `$group`, `$addFields`
4	Overall post statistics	`$group` with `$sum`, `$avg`, `$max`, `$min`
5	Most engaging posts	`$addFields`, `$multiply`, `$sort`
6	Popular interests	`$unwind`, `$group`, `$push`
7	A post with its comments	`$lookup`
8	Followers of a user in one query	two `$lookup` stages
9	Top commenters	`$group`, `$sort`
10	Activity per user	`$lookup`, `$unwind`, `$addToSet`
11	Users grouped by follower range	`$bucket`
12	Dashboard in one query	`$facet`
13	Follower ranking	`$setWindowFields`, `$rank`

7.1 Example: trending hashtags

Each post holds an array of hashtags, so the array is first split into one document per hashtag with `$unwind`, then the documents are grouped and counted.

```
db.posts.aggregate([
  { $match: { isPublic: true } },
  { $unwind: '$hashtags' },
  { $group: { _id: '$hashtags', posts: { $sum: 1 },
              totalLikes: { $sum: '$likesCount' } } },
  { $sort: { posts: -1, totalLikes: -1, _id: 1 } },
  { $limit: 5 }
])
```

The pipeline works in five steps: keep only public posts, turn a post with three hashtags into three documents, group by hashtag while counting posts and adding likes, sort so the most used hashtag comes first, and keep the top five.

7.2 Example: joining collections with $lookup

In the referenced design, `$lookup` replaces the manual two-step query. The following pipeline returns the followers of one user in a single query.

```
db.users.aggregate([
  { $match: { username: 'neha_g' } },
  { $lookup: { from: 'follows', localField: '_id',
               foreignField: 'followeeId', as: 'f' } },
  { $lookup: { from: 'users', localField: 'f.followerId',
               foreignField: '_id', as: 'followers' } },
  { $project: { _id: 0, username: 1, followersCount: 1,
                followers: '$followers.username' } }
])
```

7.3 Example: engagement score

```
db.posts.aggregate([
  { $match: { isPublic: true } },
  { $addFields: { engagementScore:
      { $add: ['$likesCount', { $multiply: ['$commentsCount', 2] }] } } },
  { $sort: { engagementScore: -1 } },
  { $limit: 3 }
])
```

Comments count double because a comment usually takes more effort than a like.

8. Results

The results below are for the seed dataset. They will differ if additional data is added.

Trending hashtags (top entries)

Hashtag	Public posts	Total likes
sunset	2	7
delhi	2	4
fashion	1	5
winter	1	5

Top users by likes received

User	Total likes
neha_g	5
meera_p	4
riya_s	3

Post statistics (public posts): 6 posts, 15 likes in total, average 2.5 likes per post, maximum 5, minimum 1.

Most engaging posts

Post	Likes	Comments	Score
Winter collection drop	5	2	9
Sunset at India Gate	3	2	7
Golden hour portrait	4	1	6

Followers of neha_g: riya_s, meera_p, sam_t, arjun_k and dev_r. This matches her stored `followersCount` of 5, which confirms that the counter and the follows collection agree.

[Insert screenshots of the script outputs and of the collections in Compass here.]

9. Testing

Test	Expected result	Mechanism
Username shorter than 3 characters	Rejected	Schema validation
Date stored as a string	Rejected	Schema validation
Negative followersCount	Rejected	Schema validation
Post without required fields	Rejected	Schema validation
Valid user document	Accepted	Schema validation
Same user likes the same post twice	Rejected (code 11000)	Unique index
Same user follows the same person twice	Rejected (code 11000)	Unique index

A further check was made by turning validation off with `validationLevel: 'off'` and re-running the tests. The invalid documents were then accepted, which confirms that the rejections come from the database validator and not from the test script.

10. Challenges and Learnings

- **Referencing is manual.** A comment's `postId` is a copy of the post's `_id`, not an automatic link. Understanding the difference between a document's own `_id` and a reference to another document's `_id` was central to the design.
- **Stale view in Compass.** Compass shows a snapshot and does not refresh itself, so after re-running the seed script the displayed IDs were out of date. Refreshing the collection, or using mongosh, which always queries live data, resolves this.
- **Validation is lost when a collection is dropped.** This determined the run order of the scripts.
- **Scripts differ from the interactive shell.** In a script file, query results are not printed automatically and must be printed with printjson, and validation errors must be caught with try/catch so that all tests run.
- **Denormalization trade-off.** Stored counters make reads fast but must be kept in step with the collections they summarize.

11. Limitations

- The dataset is small (6 users and 7 posts), so the analytics are illustrative.
- The counters and the relationship documents are updated by separate operations. If a failure occurs between them, the two can become inconsistent.
- Validation is applied after seeding, so the seed data itself is not checked by the validators. Existing documents are checked only when they are modified.
- Performance optimization, transactions and replication are not yet implemented.

12. Future Work

1. **Performance optimization.** Generate more than 100,000 documents and compare query execution before and after indexing using `explain('executionStats')`, observing the change from a collection scan to an index scan.
2. **Transactions.** Implement like/unlike and follow/unfollow as multi-document ACID transactions so that the relationship document and its counter change together or not at all.
3. **Replication.** Configure a three-node replica set and demonstrate automatic failover when the primary node is stopped. A replica set is also required for multi-document transactions.
4. **Additional indexes.** Add TTL indexes to expire old activity logs, and text indexes to search posts.

13. Conclusion

The project shows how a social media platform can be modeled and analyzed in MongoDB. Unbounded relationships such as comments, likes and follows are kept in separate collections, while small bounded data such as hashtags is embedded. Schema validation and unique indexes protect data quality inside the database itself, and the aggregation framework turns raw documents into useful analytics such as trending hashtags, top users and daily engagement. The scripted setup allows the entire database to be rebuilt and demonstrated from a clean state. The planned work on indexing, transactions and replication will extend the system toward a performant and reliable design.

References

1. MongoDB Documentation: Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/
2. MongoDB Documentation: Aggregation Pipeline. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
3. MongoDB Documentation: Schema Validation. https://www.mongodb.com/docs/manual/core/schema-validation/
4. MongoDB Documentation: Indexes. https://www.mongodb.com/docs/manual/indexes/
5. MongoDB Documentation: mongosh. https://www.mongodb.com/docs/mongodb-shell/